Sharding in Databases

Agenda

- What is Sharding?
- Sharding Mechanisms
- What does Sharding provides
- Sharding challenges and solutions
- Applications
- Citus – PostgreSQL extension
- QnA

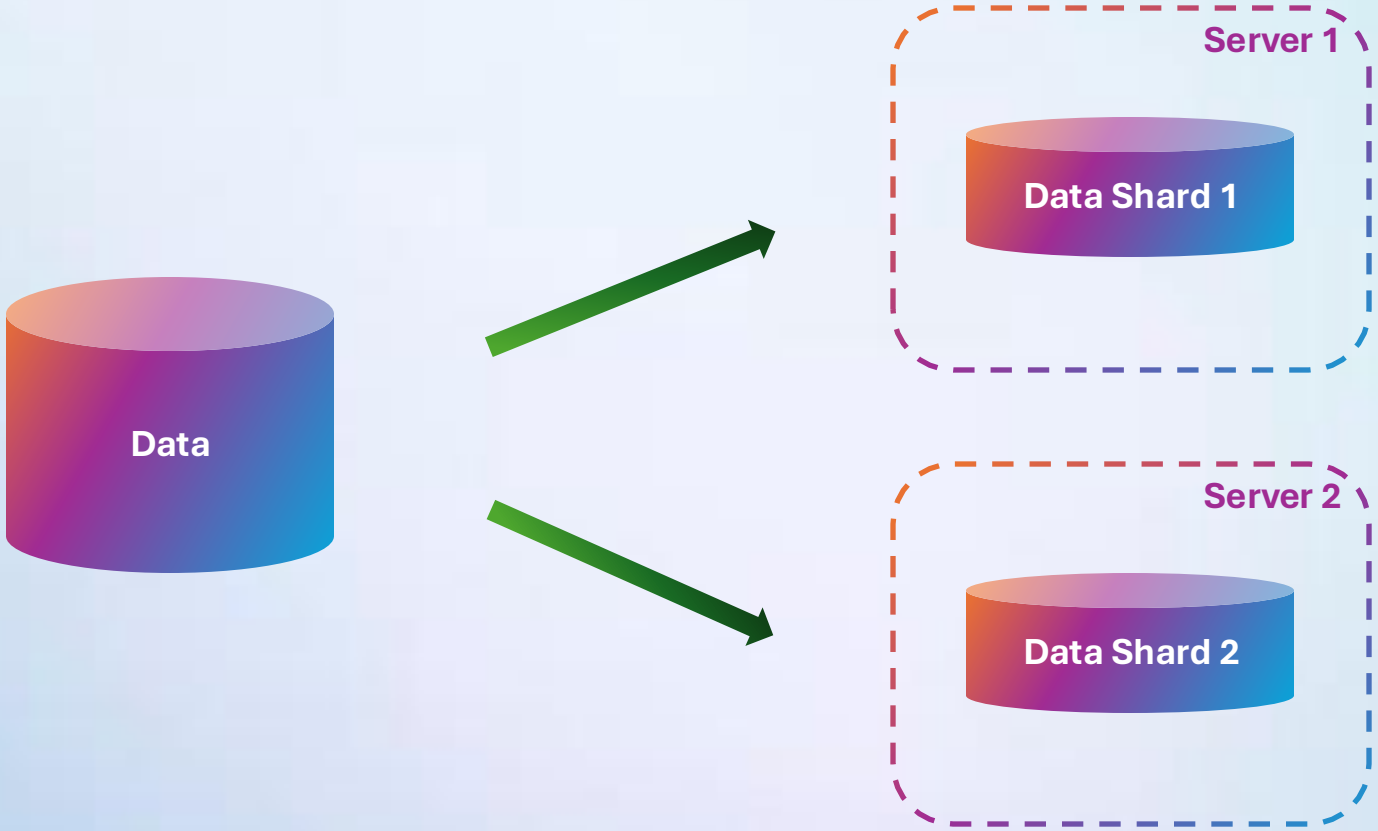
What is Sharding?

A horizontal scaling technique

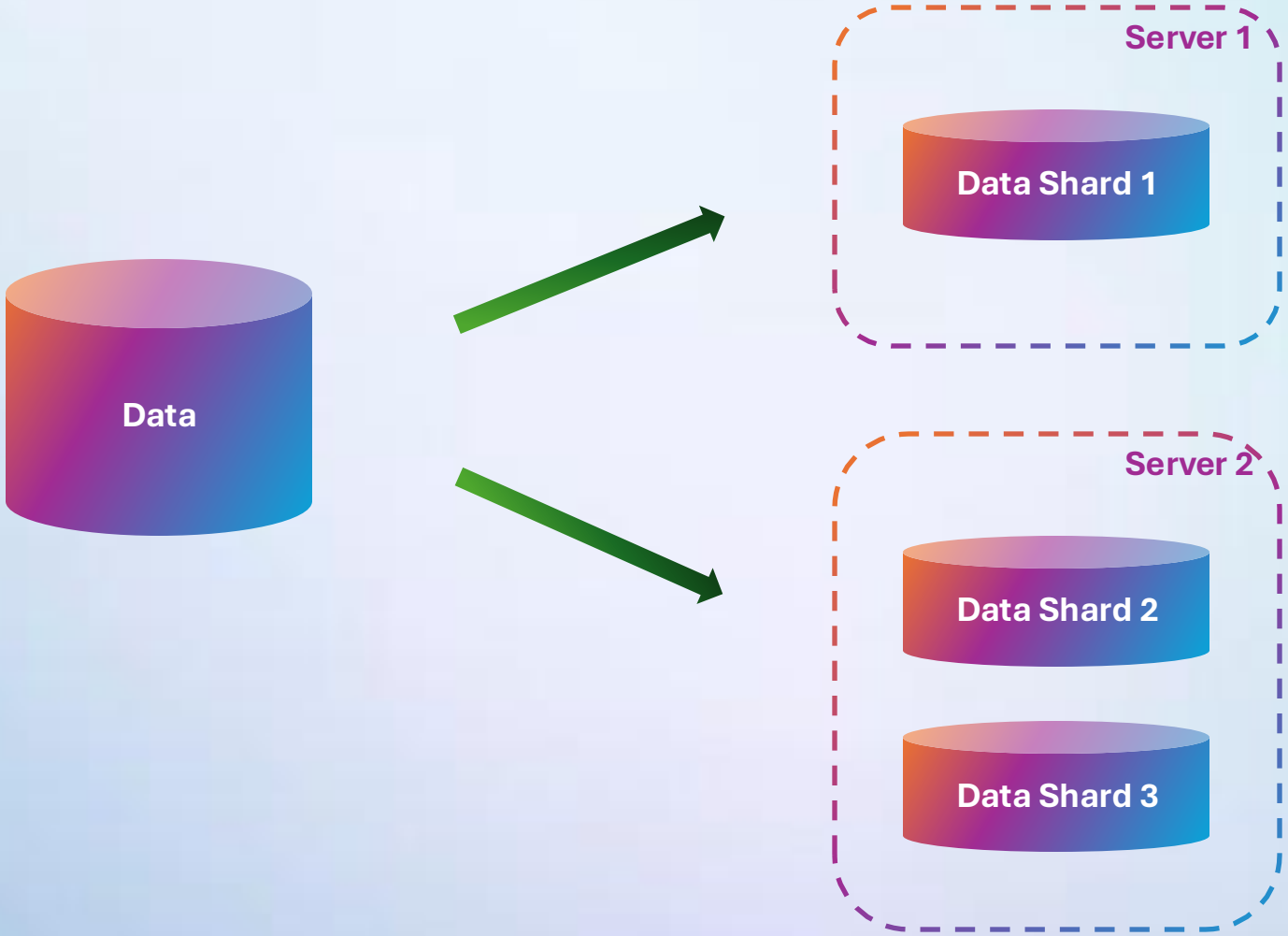
Breaks dataset into smaller - shards

Operates on a shared-nothing architecture

What is Sharding?



What is Sharding?



Is Partitioning the same thing?

Yes and No

They are both about dataset segmentation

But Partitioning is limited to one DB server

Sharding Mechanisms

How distribution works?

Each row must have a distribution key

Selected one of distribution strategies

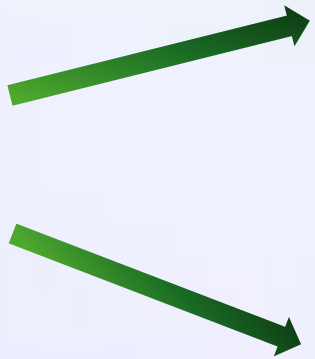
Sharding Type Groups

Vertical

Horizontal

Vertical Sharding

ID	First Name	Last Name	City
1	Alice	Anderson	Austin
2	Bob	Best	Boston
3	Carrie	Conway	Chicago
4	David	Doe	Denver



Shard 1

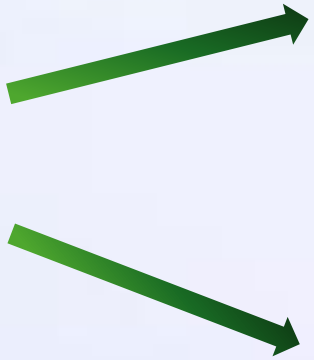
ID	City
1	Austin
2	Boston
3	Chicago
4	Denver

Shard 2

ID	First Name	Last Name
1	Alice	Anderson
2	Bob	Best
3	Carrie	Conway
4	David	Doe

Horizontal Sharding

ID	First Name	Last Name	City
1	Alice	Anderson	Austin
2	Bob	Best	Boston
3	Carrie	Conway	Chicago
4	David	Doe	Denver



Shard 1

ID	First Name	Last Name	City
1	Alice	Anderson	Austin
2	Bob	Best	Boston

Shard 2

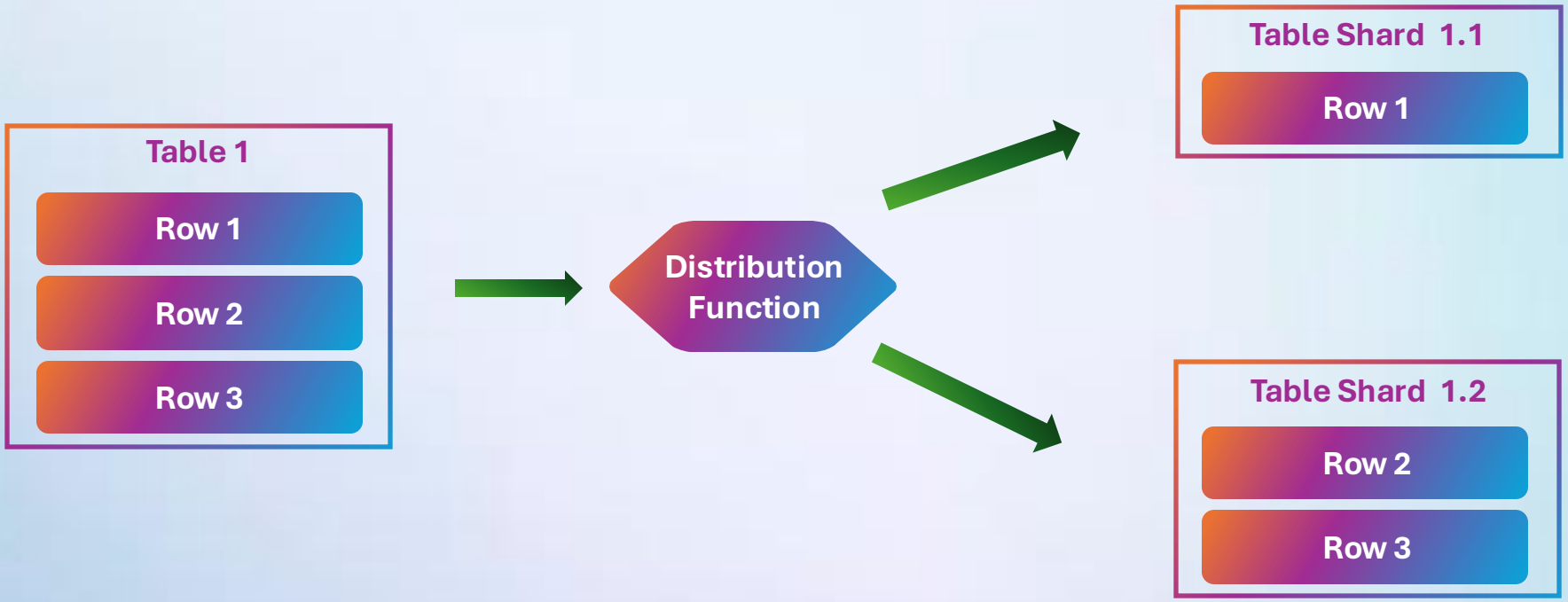
ID	First Name	Last Name	City
3	Carrie	Conway	Chicago
4	David	Doe	Denver

- Distribution column

Horizontal Sharding Types

- Range-based
- Hash-based
- Directory-based
- Geo-based

Horizontal Sharding Types



What does Sharding give us in theory?

What does Sharding give us in theory?

May improve response time

PostgreSQL has an Ace up the sleeve!

Avoid total service outage

Scale system efficiently

Main challenges

- Data hotspots
- Operation complexity
- Application complexity
- Infrastructure costs

Solving (some of) main challenges

Correctly selecting columns for distribution by Cardinality, Frequency and Monotonic change

Postpone employing Sharding

What should you try before Sharding?

Practice dictates the rules

- Optimize Data Layer
- Employ Partitioning
- Employ Vertical scaling
- Employ Replication

Practice dictates the rules

Use Database Sharding as
the last resort option

Applications of Sharding as a concept

- Multi-Tenant SaaS Database
- Real-Time Analytics
- Microservices[1]
- Distributed cache[2]

1 - https://stackoverflow.blog/2020/11/23/the-macro-problem-with-microservices/

2- https://vladmihalcea.com/jpa-hibernate-second-level-cache/

Key Takeaways

Sharding is the process of storing a large database across multiple machines

Sharding helps in situations of horizontal scaling, system tuning

Typically, sharding operates on a shared-nothing architecture

Sharding combines well with replication

Still, don't employ Sharding until you need it

Citus – a PostgreSQL extension

Core Concepts

- Focuses on Hash-based sharding
- Supports row-based and schema-based sharding
- Supports Shards Co-location, Replication
- System divides into coordinator and worker nodes
- Global consistency is eventual in some configurations
- Supports distributed transactions using two-phase commit (2PC).

Co-location

ID	First Name	Last Name
1	Alice	Anderson
2	Bob	Best
3	Carrie	Conway
4	David	Doe

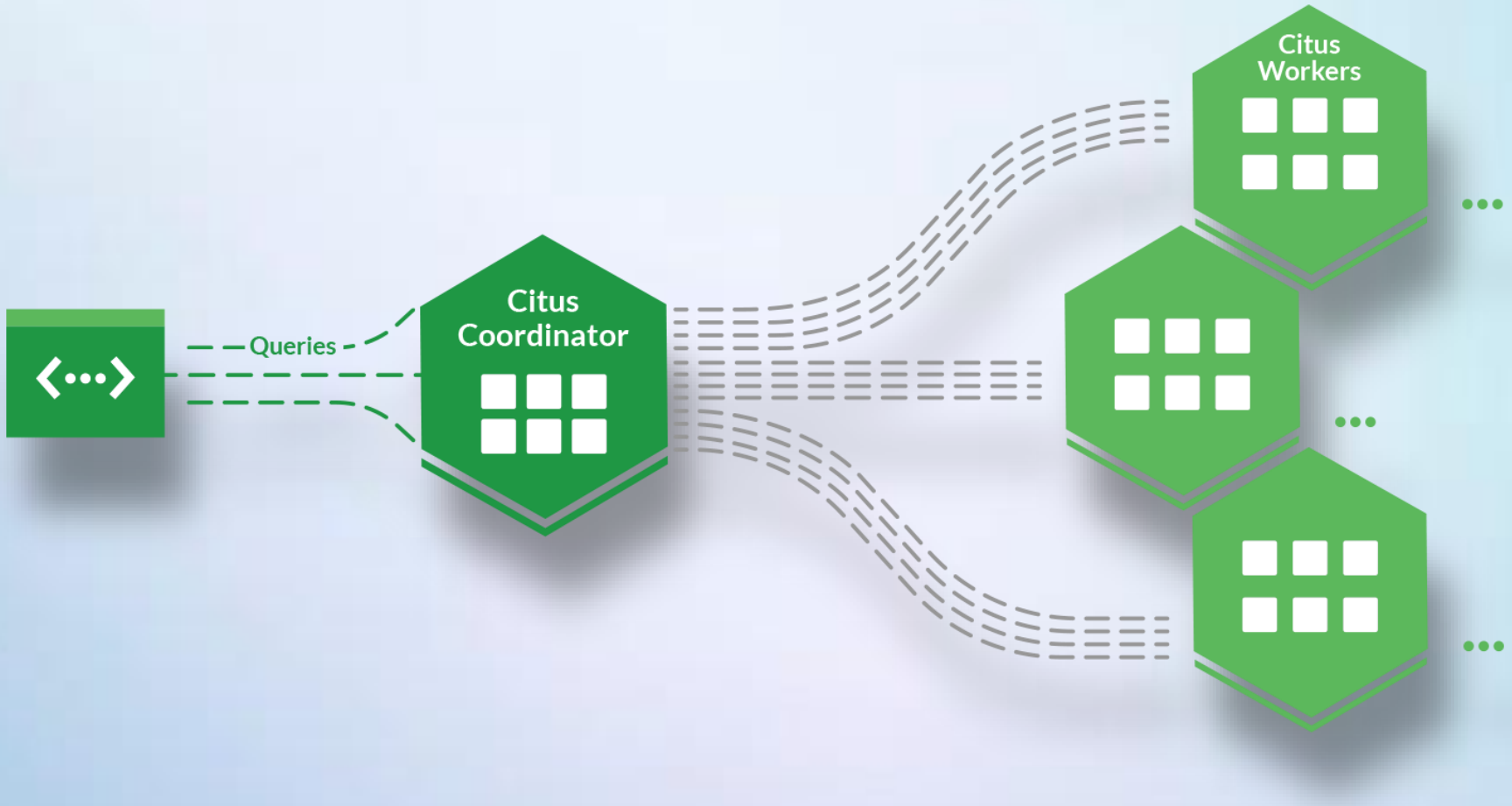
ID	Status	Client ID
101	SUCCESS	1
102	IN_PROGRESS	1
103	IN_PROGRESS	3
104	IN_PROGRESS	2

Shard 1

ID	First Name	Last Name
1	Alice	Anderson
2	Bob	Best

ID	Status	Client ID
101	SUCCESS	1
102	IN_PROGRESS	1
104	IN_PROGRESS	2

Architecture



Benchmark

Single node PosgreSQL vs 10 Citus nodes

Transactional load is simulated by HammerDB

https://www.hammerdb.com/index.html

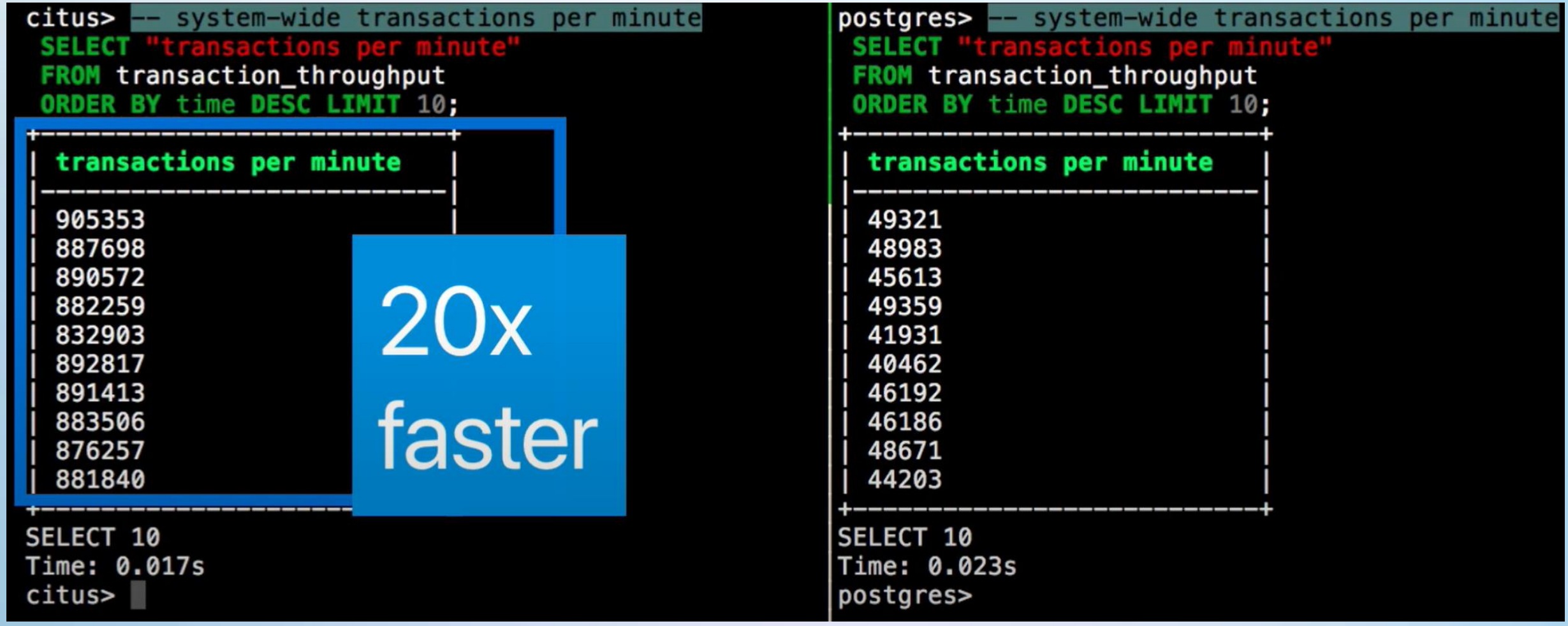
Benchmark

```
citus> -- show distributed tables and their sizes
       SELECT * FROM all_tables;
+------------------+---------+--------------------+
| table_name       | size    | distribution_col   |
|------------------+---------+--------------------|
| customer         | 39 GB   | c_w_id             |
| district         | 187 MB  | d_w_id             |
| history          | 13 GB   | h_w_id             |
| item             | 234 MB  |                    |
| nation           | 640 kB  |                    |
| new_order        | 1875 MB | no_w_id            |
| order_line       | 189 GB  | ol_w_id            |
| orders           | 16 GB   | o_w_id             |
| region           | 560 kB  |                    |
| stock            | 73 GB   | s_w_id             |
| supplier         | 24 MB   |                    |
| total_sales_rollup | 5392 kB | warehouse_id     |
| warehouse        | 48 MB   | w_id               |
| TOTAL:           | 332 GB  |                    |
+------------------+---------+--------------------+
SELECT 14
Time: 0.176s
citus>
```

```
postgres> -- show distributed tables and their sizes
          SELECT * FROM all_tables;
+----------------------------+---------+
| table_name                 | size    |
|----------------------------+---------|
| customer                   | 39 GB   |
| district                   | 80 MB   |
| history                    | 6299 MB |
| item                       | 23 MB   |
| nation                     | 56 kB   |
| new_order                  | 1275 MB |
| order_line                 | 87 GB   |
| orders                     | 7550 MB |
| region                     | 24 kB   |
| sales_by_warehouse_nation  | 16 kB   |
| stock                      | 72 GB   |
| supplier                   | 2488 kB |
| warehouse                  | 30 MB   |
| TOTAL:                     | 213 GB  |
+----------------------------+---------+
SELECT 14
Time: 1.244s (a second), executed in: 1.227s (a second)
postgres>
```

https://www.youtube.com/watch?v=W_3e07nGFxY

Benchmark



https://www.youtube.com/watch?v=W_3e07nGFxY

Benchmark

```
citus> -- analytical query on >500 million rows
 SELECT w_state, sum(ol_amount)::int
 FROM order_line JOIN warehouse ON ol_w_id = w_id
 GROUP BY w_state ORDER BY 2 DESC LIMIT 10;
+----------+----------+
| w_state  | sum      |
|----------+----------|
| IR       | 219327553 |
| 7P       | 216769763 |
| yp       | 188025085 |
| t9       | 167464242 |
| P4       | 164954825 |
| XC       | 162250242 |
| 9g       | 159572317 |
| kM       | 158577559 |
| u3       | 156267674 |
| ih       | 154380569 |
+----------+----------+
SELECT 10
Time: 10.155s (10 seconds), executed in: 10.143s (10
seconds)
```

```
postgres> -- analytical query on >500 million rows
       SELECT w_state, sum(ol_amount)::int
       FROM order_line JOIN warehouse ON ol_w_id = w_id
       GROUP BY w_state ORDER BY 2 DESC LIMIT 10;
+----------+----------+
| w_state  | sum      |
|----------+----------|
| ud       | 37938654 |
| 5d       | 37621394 |
| BK       | 35769049 |
| 5D       | 35666970 |
| pL       | 34679264 |
| pG       | 34546398 |
| Js       | 33701795 |
| QN       | 32572400 |
| Xb       | 31567706 |
| Rr       | 30316952 |
+----------+----------+
SELECT 10
Time: 3203.882s (53 minutes), executed in: 3203.869s (53 minutes)
postgres>
```

https://www.youtube.com/watch?v=W_3e07nGFxY

Citus benefits

Fully supports PostgreSQL tools and features*

Compatible with most PostgreSQL extensions

psql, ORMs, and PostgreSQL drivers
can be still used

Cloud providers like Azure provides easy setup

Analogs

Database	Range Sharding	Dictionary Sharding	Geo-Based Sharding
Citus	⚠ Emulated	✅ Hash	✅ Column-based
Vitess	✅	✅	✅
CockroachDB	✅	❌	✅
YugabyteDB	✅	✅	✅
MySQL NDB	❌	✅	❌
Azure SQL	✅	✅	✅
Spanner	✅	❌	✅
TiDB	✅	✅	⚠ Custom
Aurora	❌ (manual)	✅ (manual)	✅ (Global DB)

Analogs

Database	Range Sharding	Dictionary Sharding	Geo-Based Sharding
Citus	⚠ Emulated	✅ Hash	✅ Column-based
Vitess	✅	✅	✅
CockroachDB	✅	❌	✅
YugabyteDB	✅	✅	✅
MySQL NDB	❌	✅	❌
Azure SQL	✅	✅	✅
Spanner	✅	❌	✅
TiDB	✅	✅	⚠ Custom
Aurora	❌ (manual)	✅ (manual)	✅ (Global DB)

Key Takeaways

Citus is an open-source hash-based Sharding solution, supporting replication

Citus is a PostgreSQL extension – supports (almost) everything that PostgreSQL supports

Citus has strong tools for analytics - rollup table mechanism

Thank you

- Author: Serhii Kravchuk
- Find me in LinkedIn
- Date: June 2025
- Join Codeus community in Discord
- Join Codeus community in LinkedIn